

State-of-the-Art Note

Agentic Geospatial Reasoning for Precision-Agriculture Decision Support

DeepShift AI Summer Internship 2026 — Project 12 (Agriculture)

Mid-2026

1. Executive Summary

Precision agriculture is undergoing the same architectural transition that has reshaped enterprise software over the past two years: a shift from static, single-purpose data pipelines toward *agentic* systems capable of autonomously retrieving, reasoning over, and acting on heterogeneous streams of data. Until recently, irrigation and fertilization decision tools were built as fixed pipelines — a satellite index computed here, a weather forecast pulled there, thresholds hard-coded in between. Such systems are brittle: they cannot explain themselves, cannot adapt their reasoning to an unusual combination of signals (e.g., a false NDVI dip caused by cloud shadow rather than plant stress), and offer growers little basis for trust.

The project positioned by this note departs from that paradigm. It treats irrigation and fertilization timing as a *multi-step reasoning problem* to be solved by a stateful agent that fuses three heterogeneous modalities — optical remote sensing (Sentinel-2 vegetation indices), numerical weather and soil forecasts (Open-Meteo), and ground or simulated IoT soil-moisture streams — and produces a recommendation that is not merely accurate but *defensible*: traceable, step by step, back to the input evidence that justified it. The goal is not simply to forecast a number, but to close the loop from raw multimodal signal to an actionable, explainable recommendation that a grower or agronomist can audit before acting on it. This note surveys the 2026 technological baseline against which the project’s design choices are made, identifies the gaps that motivate an agentic (rather than purely predictive) architecture, and lays out the resulting system design, benchmark strategy, and trade-offs.

2. Technological Landscape (2026 Baseline)

2.1 Geospatial Data: From Legacy Rasters to Cloud-Native STAC Workflows

Legacy remote-sensing pipelines relied on manually downloaded, locally stored GeoTIFF tiles, with cloud masking, reprojection, and mosaicking implemented as bespoke, non-reusable scripts. This approach does not scale to an agent that must decide, at run time, *which* imagery to pull for a given field and date range. The 2026 baseline instead standardizes around the SpatioTemporal Asset Catalog (STAC) specification, which exposes satellite archives as queryable, filterable catalogs of georeferenced assets rather than opaque file dumps. The Copernicus Data Space Ecosystem exposes Sentinel-2 Level-2A (atmospherically corrected, analysis-ready) imagery through a STAC-compliant API at no cost, while Google Earth Engine offers the complementary advantage of server-side processing — computing NDVI/EVI/NDWI composites in the cloud rather than requiring local download of full scenes. For an agentic system, this distinction matters directly: a STAC-aware tool call can be treated as a discrete, composable action in an agent’s tool palette (*search catalog* → *filter by cloud cover* → *compute index*), whereas legacy raster workflows resist this decomposition. The project’s data-access layer is therefore built on `pystac-client` against the Copernicus STAC endpoint, with Earth Engine as a secondary, convenience-oriented path via `geemap`.

2.2 Time-Series Forecasting: From LSTM/ARIMA to Zero-Shot Foundation Models

Classical forecasting for soil moisture and crop water demand has relied on either statistical models (ARIMA, exponential smoothing) or supervised deep-learning architectures (LSTM, and more recently Temporal Fusion Transformer and N-HiTS), all of which require a non-trivial history of labeled, field-specific data to fit well. This is precisely what most individual farms lack: a single field may have only a season or two of sensor history, sensor dropouts are common, and crop-season variability means the training distribution shifts year to year. Through 2025, time-series *foundation models* closed much of this gap. Amazon’s Chronos-2, released in October 2025, tokenizes time-series values through scaling and

quantization and treats forecasting as a sequence-modeling problem in the style of a language model; critically, its October 2025 revision extended the original univariate design to natively support multivariate and covariate-informed forecasting, allowing weather and soil covariates to condition the forecast without any task-specific fine-tuning. Google’s TimesFM and the probabilistic Lag-Llama occupy adjacent positions in the same family. The practical benefit for this project is that a short-horizon soil-moisture or crop-water-need forecast can be produced in a zero-shot or few-shot regime directly from a short context window pulled from Open-Meteo and IoT streams, without maintaining a per-field training pipeline. TFT and N-HiTS are retained not as the primary forecaster but as supervised *baselines*: their role is to quantify how much accuracy, if any, is sacrificed by avoiding field-specific training, and to serve as a sanity check against foundation-model failure modes such as poor extrapolation under regime shifts (e.g., an unseasonal heatwave).

2.3 Agentic Reasoning: From Linear Chains to Cyclic, Stateful Graphs

The earlier generation of “LLM pipeline” tools — typified by simple prompt chains — executes a fixed, linear sequence of calls: retrieve, summarize, generate. This structure cannot represent the kind of reasoning irrigation decisions actually require, where the agent may need to re-query a data source after an intermediate result looks anomalous, or branch its reasoning depending on whether a conflict is detected between the vegetation-index trend and the soil-moisture forecast. LangGraph addresses this by modeling the agent as a cyclic, stateful graph rather than a linear chain: nodes represent reasoning or tool-call steps, edges represent control flow, and a persistent state object carries intermediate results (retrieved indices, forecasts, partial explanations) across the graph. Implemented as a ReAct-style loop — the agent alternates between reasoning about what it knows and calling a tool to obtain what it doesn’t — this pattern allows the agent to re-plan, retry, or seek additional evidence mid-task rather than committing to a single fixed sequence. This is the orchestration substrate mandated for the fusion agent in this project, and it is also the substrate used by comparable published systems, including the GeoBenchX reference agent itself, which is built as a LangGraph ReAct loop over a palette of geospatial tools.

3. Gap Analysis in Precision Agriculture

3.1 The Fragmentation Problem

Satellite indices, numerical weather forecasts, and IoT soil-moisture streams are fundamentally misaligned along three axes: *spatial* (a 10 m Sentinel-2 pixel grid versus point-source sensor readings versus a coarser weather-model grid), *temporal* (a five-day satellite revisit cycle versus hourly weather forecasts versus continuous or sub-hourly IoT telemetry), and *semantic* (a vegetation index is a unitless proxy for canopy health, while soil moisture is a volumetric physical quantity, and the two do not share a common scale). Treating fusion as an afterthought — a naive join or a fixed-window average — discards exactly the information a defensible recommendation needs: whether a stress signal in one modality is corroborated by, or contradicted by, another. This is the fragmentation problem, and it is the primary technical justification for a fusion layer that aligns data *before* any forecasting or reasoning is attempted, rather than fusing outputs after the fact.

3.2 The Black-Box Challenge

A recommendation engine that outputs “irrigate field 4 with 12mm” without exposing the evidence behind that number is not adoptable by a working grower, who bears the operational and financial consequences of a wrong call. Deep supervised forecasters and even zero-shot foundation models are, by construction, difficult to interrogate post hoc: a TFT attention map or a Chronos-2 quantile forecast does not by itself constitute an explanation a non-specialist can evaluate. Interpretability is therefore not a cosmetic feature layered on top of the system; it must be a first-class design constraint governing how signals are combined and how the reasoning trace is surfaced.

3.3 Defensibility as a Design Requirement

Defensibility goes one step beyond interpretability: the system must be able to show *which* inputs it consulted, *what* each one indicated, and *how* they were weighed against each other, in a form that survives scrutiny. This project treats the LangGraph state trace — the ordered record of tool calls, intermediate values, and the agent’s stated reasoning at each step — as the substrate for defensibility, rather than attempting to retrofit an explanation onto an opaque final output.

4. Architectural Framework

4.1 Data Alignment with `xarray`

The fusion layer standardizes all inputs — Sentinel-2-derived indices, Open-Meteo weather and soil-moisture grids, and IoT time series — onto a common labeled, multi-dimensional array structure using `xarray`. Unlike a plain NumPy array or a flat DataFrame, an `xarray.Dataset` carries explicit coordinate labels for time, latitude, and longitude, which makes spatial and temporal alignment (reprojection, resampling, nearest-neighbor or interpolated joins across mismatched grids) an explicit, auditable operation rather than an implicit one buried in index arithmetic. This directly addresses the fragmentation problem described in Section 3.1: once all three modalities live in a common `xarray` coordinate system, downstream forecasting and agent tool calls can query a single consistent data structure rather than reconciling three incompatible schemas on every call.

4.2 Hybrid Forecasting Strategy

The forecasting layer follows a hybrid strategy rather than committing to a single model family. Chronos-2 (or TimesFM as an alternative) is used as the primary, zero/few-shot forecaster, chosen specifically because most individual fields will not have enough historical data to train a supervised model well, and because a foundation model’s covariate-conditioning capability allows weather and satellite-derived covariates to inform the soil-moisture forecast without a separate fusion step at the forecasting stage. TFT and N-HiTS are trained as supervised baselines on whatever field history is available and used strictly as a validation reference: the gap between the zero-shot foundation-model forecast and the supervised baseline is itself a diagnostic — a large gap signals that the field in question may be poorly represented in the foundation model’s pretraining distribution and that its recommendations should be treated with additional caution.

4.3 The Reasoning Loop: Separating the Deterministic Pipeline from the Agent

Not every step here benefits from being agent-controlled. Retrieval, spatial-temporal alignment, and dual-model forecasting (Section 4.2) run as a fixed, deterministic pipeline: given a field and a date, these steps execute identically every cycle regardless of what the data shows — there is no question of *whether* to run the baseline comparison, since the gap between the zero-shot and supervised forecasts is itself a diagnostic. This mirrors the project’s own timeline: forecasting is a Weeks 3–4 deliverable, validated in isolation before the agent is built in Weeks 5–6.

The agent’s loop begins only once the pipeline has produced a forecast bundle, and alternates between two nodes: an *agent* node, where the LLM reasons over the forecast alongside the raw satellite, weather, and soil signals and decides whether the evidence is sufficient to recommend an action; and a *tools* node, invoked only when that reasoning surfaces a gap or contradiction — e.g., a soil-moisture forecast disagreeing with the NDVI trend — and needs to re-query a wider weather window or re-check data quality before proceeding. Each pass through the tools node returns control to the agent node with updated state, so the loop terminates once the agent judges the fused evidence adequate and synthesizes it into an action with a step-referenced explanation. This is the system’s only genuinely agentic decision point: everything upstream is pipeline, and everything downstream follows from the agent’s own reasoning trace.

5. Benchmark Strategy: GeoBenchX

Standard forecasting accuracy metrics (RMSE, MASE) evaluate only the numerical forecasting sub-component of the system; they say nothing about whether the *agent* correctly decided which tools to

call, in which order, or whether it correctly recognized when a task was unsolvable given the available evidence — both of which are central to whether the system can be trusted operationally. GeoBenchX, a published benchmark evaluating tool-calling LLM agents on multi-step geospatial tasks using a LangGraph ReAct agent equipped with geospatial functions, provides a task methodology directly applicable here: multi-step tasks of increasing complexity, evaluated with an LLM-as-judge protocol that compares an agent’s tool-call trace and final output against a reference solution, including tasks that are intentionally unsolvable to test whether the agent over-commits to an answer rather than correctly declining. This project adopts the same evaluation posture for its own task set: constructing field-specific multi-step scenarios (e.g., “determine whether field 7 requires irrigation given a partial sensor dropout”) and scoring the agent on *tool-call correctness* (did it query the right sources, in a sensible order) and *plan completion* (did it reach a complete, evidence-grounded recommendation rather than stalling or hallucinating a result), in addition to conventional decision-accuracy metrics against agronomist-validated ground truth.

6. Critical Design Trade-offs

6.1 Compute Overhead versus Real-Time Latency

A multi-step ReAct agent that issues several sequential tool calls — STAC query, weather fetch, forecast inference, explanation synthesis — necessarily incurs higher latency than a single fixed-pipeline inference pass. For irrigation and fertilization decisions, which operate on a daily-to-weekly cadence rather than a sub-second one, this overhead is acceptable and arguably a reasonable price for defensibility; the project does not target real-time (sub-minute) responsiveness, but does cap the number of agent iterations per task and caches STAC/weather query results to bound cost and avoid redundant tool calls within a single decision cycle.

6.2 Temporal Splitting to Prevent Data Leakage

Because the forecasting and evaluation data are drawn from continuous, autocorrelated time series, a naive random train/test split would leak future information into training (a model could, in effect, “see the future” via temporally adjacent samples) and produce an artificially optimistic accuracy estimate. All forecasting evaluation in this project therefore uses strict temporal (chronological) splitting, with the validation and test windows drawn from periods strictly after the training window, and with any synthetic or foundation-model-imputed data generated only from information available up to the split point — a discipline that is especially important given that Chronos-2 and comparable foundation models are evaluated here in a zero-shot regime, where the absence of task-specific training makes it easy to understate the risk of leakage through the covariate or context window rather than through the training set itself.

7. Conclusion & Roadmap

The system architecture surveyed in this note — STAC-native geospatial access, zero-shot time-series foundation models validated against supervised baselines, and a stateful LangGraph reasoning loop — is not simply an assemblage of currently fashionable components; each choice is a direct response to a specific gap identified in Section 3: `xarray`-based alignment answers the fragmentation problem, the hybrid forecasting strategy answers the sparse-data constraint that makes purely supervised approaches unreliable at the field level, and the GeoBenchX-style evaluation protocol answers the black-box and defensibility challenges that would otherwise leave the system’s recommendations unauditably. It is worth stating plainly where the approach remains fragile: foundation-model forecasts are only as reliable as their pretraining distribution’s coverage of agricultural time series, which is not yet well characterized publicly; LLM-as-judge evaluation is itself an imperfect proxy for ground truth and can be gamed by superficially plausible but substantively wrong reasoning traces; and the reliance on simulated IoT data where real sensors are unavailable means validation against true field outcomes remains the binding constraint on how much confidence the system’s recommendations can ultimately claim. The eight-week roadmap treats these limitations as first-class risks to be measured, not assumptions taken on faith: every later claim of forecast accuracy, tool-call correctness, or recommendation quality will be benchmarked against Sections 5 and 6’s criteria, using the temporally disciplined protocol established here.